

TECHSHOT

# Minimal APIs na prática

Construindo APIs mais simples e eficientes

APRESENTADO POR

**Artur Bomtempo**

DATA

**Agosto de 2026**



Quem é

# Artur Bomtempo

- Desenvolvedor de Software na dti digital
- Estudante de Engenharia de Software na PUC Minas
- Criador de conteúdo sobre tecnologia
- Chapter Lead no WebTech Network



## SUMÁRIO

### 01 Fundamentos

Entenda o que são Minimal APIs, sua origem e os motivos por trás dessa abordagem.

### 02 Funcionamento

Explore como uma Minimal API funciona e como organizar suas principais responsabilidades.

### 03 Hands-on

Coloque os conceitos em prática construindo uma API .NET do zero.



# 01

# Fundamentos

Entenda o que são Minimal APIs, sua origem e os motivos por trás dessa abordagem.

# Antes das Minimal APIs

---

## APIs tradicionais

As APIs em ASP.NET Core tradicionalmente utilizavam Controllers.

---

## Rotas e convenções

Rotas e comportamentos eram definidos por classes, atributos e convenções.

---

## Mais cerimônia

A estrutura do framework trazia mais abstrações e cerimônias para a construção das APIs.

---

## O surgimento das Minimal APIs

Com o .NET 6, as Minimal APIs surgiram como uma alternativa mais enxuta.

# O PROBLEMA

Por que buscar uma nova abordagem?

---

## Estrutura nem sempre necessária

Nem toda API precisa de toda a estrutura oferecida pelo modelo tradicional.

---

## Mais complexidade

Classes, atributos e convenções podem tornar a construção de endpoints mais trabalhosa.

---

## Menos explicitude

O caminho entre uma rota HTTP e seu handler pode ficar menos explícito.

---

## Uma experiência mais simples

Para APIs focadas em endpoints HTTP, podemos simplificar essa experiência.



COMPARAÇÃO

# Controller x Minimal API

| ASPECTO            | CONTROLLER                             | MINIMAL API                        |
|--------------------|----------------------------------------|------------------------------------|
| <b>Rotas</b>       | Atributos e convenções                 | Declaradas explicitamente          |
| <b>Handler</b>     | Método dentro de um Controller         | Handler associado ao endpoint      |
| <b>Organização</b> | Estrutura mais definida pelo framework | Maior liberdade de organização     |
| <b>Entrada</b>     | Controllers descobertos pelo framework | Endpoints registrados na aplicação |
| <b>Estrutura</b>   | Mais abstrações                        | Menos abstrações                   |

## HISTÓRICO

# A evolução das APIs no .NET

Das APIs baseadas em Controllers à abordagem mais enxuta das Minimal APIs.

ATÉ .NET 5

## Modelo tradicional

APIs eram tradicionalmente construídas utilizando Controllers, atributos e convenções.

.NET 6

## Surgimento das Minimal APIs

O .NET introduziu as Minimal APIs como uma alternativa com menos código e configuração.

.NET 7-9

## Evolução

As Minimal APIs continuam evoluindo dentro do ASP.NET Core e ganham espaço em aplicações reais.

HOJE

## Maturidade

Minimal APIs se consolidaram como uma opção para aplicações reais, inclusive projetos maiores e mais complexos.



**MAS AFINAL... O QUE  
É UMA MINIMAL API?**

# MENOS ESTRUTURA. MAIS CÓDIGO DIRETO.

Minimal APIs são uma abordagem do ASP.NET Core para criar APIs HTTP com menos código, abstrações e estruturas desnecessárias.

## **Rotas no código**

MapGet, MapPost, MapPut, MapDelete

As rotas são definidas diretamente no código.

## **Sem Controllers**

Os endpoints não precisam da estrutura tradicional de Controllers e Actions.

## **Fluxo direto**

Endpoint → lógica → resposta

O fluxo da aplicação fica mais simples e explícito.



“MINIMAL” NÃO SIGNIFICA PEQUENO

**Minimal se refere à quantidade de estrutura e código necessários para construir a API, não ao tamanho da aplicação.**

**MAS SE NÃO TEMOS  
CONTROLLER... QUEM  
ORGANIZA A APLICAÇÃO?**



# Nós.

A Minimal API **não define uma estrutura de projeto obrigatória.**

Nós escolhemos como organizar os endpoints, regras de negócio e responsabilidades da aplicação.

# 02

## Funcionamento

Explore como uma Minimal API funciona e como organizar suas principais responsabilidades.

## FUNIONAMENTO

# Como uma requisição percorre a aplicação

Uma visão simplificada da estrutura que utilizaremos para construir nossa Minimal API.



**Não existe uma estrutura única para Minimal APIs. Esta é uma organização que utilizo no dia a dia e recomendo.**



# Responsabilidade de cada módulo

| MÓDULO    | RESPONSABILIDADE                          |
|-----------|-------------------------------------------|
| Module    | Rotas, HTTP e status codes                |
| Contract  | Contratos e comunicação entre componentes |
| Aggregate | Entidades e regras de negócio             |
| Schemas   | Entrada, saída e validação dos dados      |
| Data      | Persistência e acesso aos dados           |
| IoC       | Injeção de dependências e configurações   |

*Cada módulo possui uma responsabilidade clara e juntos formam a estrutura da aplicação.*

A REGRA DE OURO

**O Module não conhece o banco.  
O Aggregate não conhece HTTP.**

Cada parte conhece apenas o que precisa para cumprir sua responsabilidade.

# 03

## Hands-on

Coloque os conceitos em prática construindo uma API .NET do zero.



# O que vamos construir?

## TuneTrail

Uma API de diário musical construída com .NET Minimal API para registrar e consultar músicas ouvidas.

## FERRAMENTA

## USO

**.NET 10 SDK**

Desenvolvimento da API

**Visual Studio Code**

Editor de código

**PostgreSQL**

Banco de dados

**Docker Desktop**

Executar o PostgreSQL

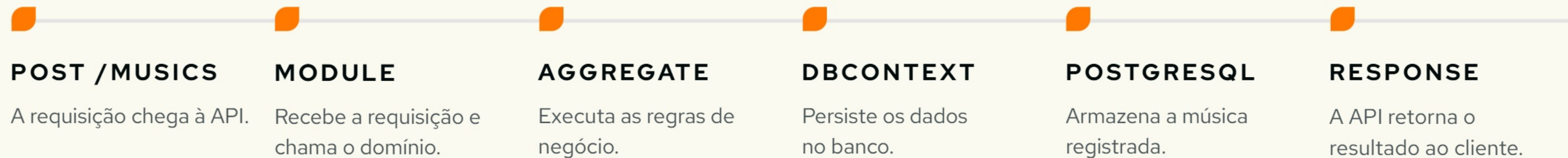
**Reqly (*opcional*)**

Testar os endpoints

O CAMINHO DE UMA REQUISIÇÃO

# O fluxo que vamos construir

Vamos acompanhar o que acontece quando uma música é registrada na API.



# **ALGUMAS COISAS PARA TER EM MENTE**



# ENTITY ≠ API CONTRACT

A Entity representa o modelo interno da aplicação.

O Contract representa o que a API expõe.

---

## Segurança

Não exponha dados internos desnecessariamente.

---

## Desacoplamento

A API não depende diretamente da estrutura da Entity.

---

## Flexibilidade

Request e Response podem ter formatos diferentes.

---

## Evolução

A Entity pode mudar sem necessariamente alterar o contrato da API.

RESULT PATTERN

# E QUANDO UMA OPERAÇÃO FALHA?

Nem toda falha precisa ser uma exceção.

SUCESSO

**Result → Data**

A operação foi concluída e retorna os dados esperados.

FALHA ESPERADA

**Result → Code + Message**

A operação não pôde ser concluída e retorna uma informação sobre o motivo.

# Como as peças se encontram?



## Module

Depende de **IMusicAggregate** - conhece apenas o contrato.



## IMusicAggregate

Define o **contrato do domínio** - expõe as operações disponíveis.



## MusicAggregate

Implementa as **regras de negócio** - concentra a lógica do domínio.



## TuneTrailDbContext

**Acessa os dados** - comunica-se com o banco.



# E o Program.cs?

Ele deve ser um índice, não um depósito.

## CONFIGURAÇÃO

### Prepara a aplicação

Banco, JSON e Swagger.

## SERVIÇOS

### Registra as dependências

Centraliza o `RegisterServices()` .

## MÓDULOS

### Registra os endpoints

Conecta os módulos com `RegisterModules()` .

## APLICAÇÃO

### Monta e executa a API

Cria o `app` , configura o pipeline e inicia.



**Falar é fácil,  
me mostre o código.**

**Linus Torvalds**

Criador e Mantenedor do Linux

# Vamos conversar?

CONTATO

[artur.bomtempo@dtidigital.com.br](mailto:artur.bomtempo@dtidigital.com.br)

LINKEDIN

[linkedin.com/in/artur-bomtempo](https://www.linkedin.com/in/artur-bomtempo)